

Programación Avanzada

2026

Prof. Martha Gimenez

ENTREGABLE 3 - DSL

Juan Martín Berglavaz

Camila García

Índice

Introducción..... 2
Decisiones de diseño e implementación tomadas..... 3
Herramientas utilizadas..... 8
1. Patrón de Diseño Builder (DSL Interno)..... 8
2. Lark para Análisis Sintáctico (DSL Externo):..... 8
3. Pytest para Pruebas Unitarias..... 8
4. Docker..... 8
Reglas de producción..... 9
1. Gramática del DSL de Mantenimiento..... 9
Decisiones clave de esta gramática:..... 10
2. Gramática del DSL de Compras..... 10
Decisiones clave de esta gramática:..... 11
Conclusiones..... 11

Introducción

Este informe detalla el diseño, la arquitectura y la implementación de un sistema de gestión para un supermercado. El objetivo principal del proyecto es aplicar los conceptos de Lenguajes Específicos de Dominio (DSL), tanto internos como externos, y demostrar el uso de patrones de diseño avanzados para resolver un problema de negocio real.

El sistema desarrollado permite la carga inicial de productos y promociones, la gestión (alta, baja y modificación) de dichos datos, y la realización de compras aplicando reglas de negocio para el cálculo de descuentos. Para lograrlo, se diseñó una arquitectura desacoplada. El Modelo Semántico del dominio está centralizado en un servicio que actúa como Facade (única puerta de entrada a la lógica de negocio), ocultando la estructura de datos interna.

Para la interacción del usuario y la carga de datos, se implementaron dos enfoques distintos:

1. **DSL Interno (Fluent Interface):** Utilizado para la configuración inicial del supermercado, permitiendo encadenar métodos de forma declarativa y legible (ej. dsl.producto(1).nombre("Arroz")...), ocultando la complejidad de la instanciación de objetos mediante el patrón Expression Builder.
2. **DSL Externos (Parsers):** Implementados para la gestión de mantenimiento (actualización de productos/promociones) y la realización de compras, utilizando la herramienta de parsing Lark para construir ambos lenguajes a partir de gramáticas EBNF. Estos lenguajes externos permiten a un usuario escribir instrucciones en archivos de texto plano con una sintaxis propia, las cuales son interpretadas y validadas por el sistema.

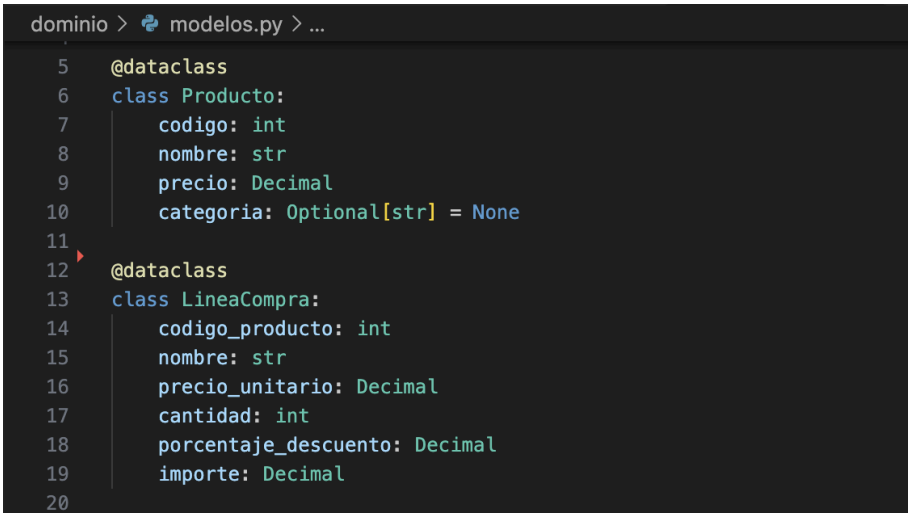
Un aspecto fundamental del diseño fue la separación de responsabilidades. Mientras que los parsers (DSL externos) se encargan de la sintaxis y de construir un Árbol de Sintaxis Abstracta (AST), el modelo de dominio se encarga de la semántica (por ejemplo validar que los productos existan, que no haya códigos duplicados, etc.) y de la lógica algorítmica. Esta división permite que el sistema no aborte ante errores semánticos, sino que los capture, continúe procesando el resto del archivo y devuelva una lista de errores al usuario.

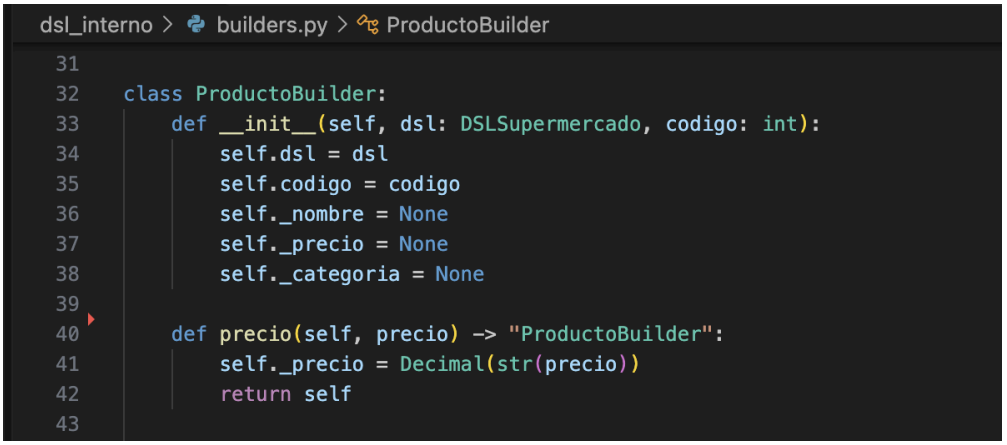
Para la aplicación de descuentos en las compras, se desarrolló un motor independiente basado en un enfoque Greedy, capaz de resolver múltiples combinaciones de promociones aplicando la mejor opción disponible en cada paso y respetando la precisión matemática mediante el uso de la clase Decimal.

Decisiones de diseño e implementación tomadas

1. **Uso de Decimal en lugar de float para el manejo del dinero**

   Decidimos usar la clase Decimal de la librería estándar de Python para todos los precios, descuentos e importes. Si bien float es el tipo de dato por defecto para números con coma, tiene un problema de precisión. En un sistema de supermercado, los errores de centavos son inaceptables. Con Decimal y el redondeo ROUND_HALF_UP, nos aseguramos de que los cálculos monetarios sean exactos.



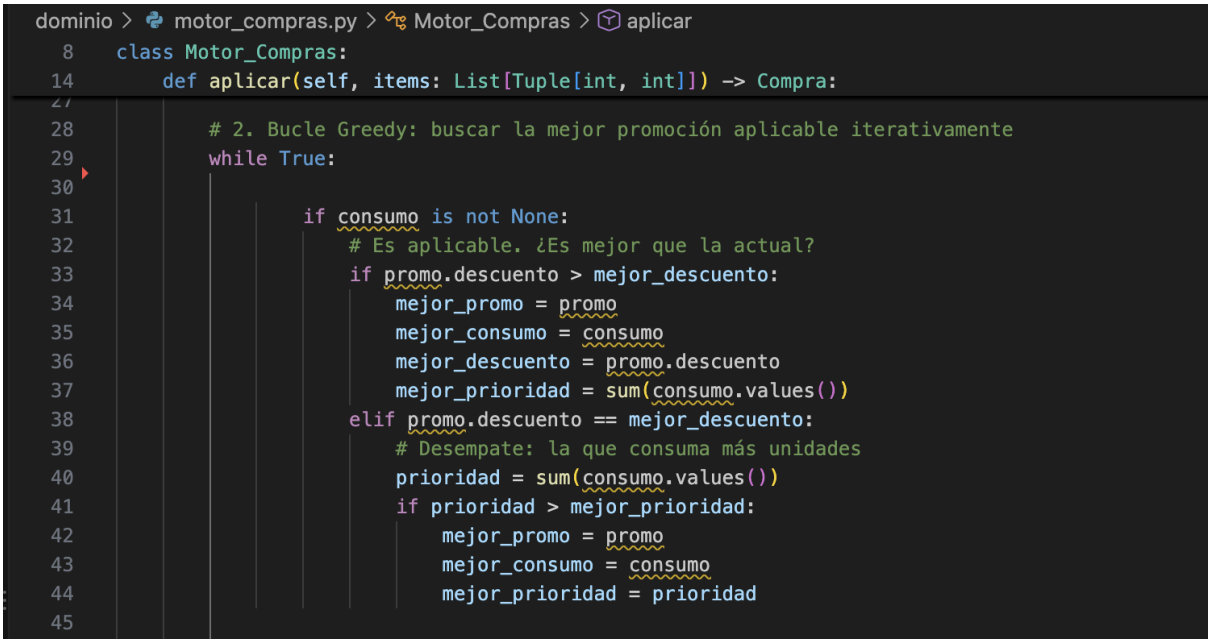


2. Implementación del Motor de Compras con enfoque Greedy

Se diseñó un módulo independiente (motor_compras) que resuelve la aplicación de promociones mediante un enfoque algorítmico Greedy iterativo. Este evalúa las promociones aplicables, selecciona la de mayor porcentaje de descuento, consume el stock involucrado y repite el proceso con el resto.

Como la consigna pide que ante múltiples promociones aplicables se priorice la que genere el mayor porcentaje de descuento, permitiendo que un mismo producto reciba distintos descuentos en diferentes unidades, un enfoque de fuerza bruta resultaría ineficiente. El algoritmo Greedy resuelve el problema de manera óptima y con una complejidad computacional adecuada.

Adicionalmente, se implementó un criterio de desempate determinista (que prioriza la promoción que consuma mayor cantidad de unidades) para garantizar la reproducibilidad de los resultados.



3. Aplicación del patrón de diseño Facade (Fachada) para el modelo de dominio

Se centralizó toda la lógica de negocio y validación semántica en la clase SupermercadoService, la cual actúa como única interfaz de acceso al modelo de dominio. Este diseño funciona como una API de Comando-Consulta (los métodos de actualización modifican el estado y las consultas devuelven información sin modificarlo), ocultando la estructura de datos interna (diccionarios) a los clientes externos.

Este desacoplamiento permite que tanto el DSL interno (Expression Builders) como los DSL externos (Parsers) interactúen con el sistema sin conocer los detalles de implementación. Asimismo, garantiza que todas las reglas de negocio (unicidad de códigos, integridad referencial, obligatoriedad de campos) se validen en un único punto.

dominio > servicio.py > ...

```python
class SupermercadoService:
    def __init__(self):
        # Usamos diccionarios para búsquedas rápidas por código
        self._productos: dict[int, Producto] = {}
        self._promociones: dict[int, Promocion] = {}
        self._compras: List[Compra] = []

    # --- PRODUCTOS ---
    def agregar_producto(self, codigo: int, nombre: str, precio: Decimal, categoria: Optional[str] = None) -> None:
        self._validar_entero(codigo, "código")
        if codigo in self._productos:
            raise ErrorSemantico(f"El producto con código {codigo} ya existe.")

        self._validar_texto(nombre, "nombre")
        self._validar_decimal(precio, "precio")
        if categoria:
            self._validar_texto(categoria, "categoría")

        self._productos[codigo] = Producto(codigo, nombre, precio, categoria)

    # --- LISTADOS ---
    def listar_productos(self) -> List[Producto]:
        return list(self._productos.values())
```

4. **Utilización de @dataclass y del patrón Builder con Fluent Interface para el DSL Interno**

Se usaron dataclasses para la definición de las entidades del dominio y se implementó el patrón Expression Builder junto con una Fluent Interface para la configuración inicial del supermercado.

Las dataclasses minimizan el código repetitivo asociado a la inicialización de objetos. Por su parte, el patrón Expression Builder permite construir objetos complejos paso a paso aislando la lógica de construcción. Al integrarlo con una Fluent Interface (encadenamiento de métodos), se logra un código declarativo y altamente legible. La implementación realiza un commit automático contra el servicio al cambiar de contexto, asegurando la persistencia de los datos de manera transparente para el usuario.

dominio > modelos.py > ...

```python
@dataclass
class Promocion:
    codigo: int
    nombre: str
    descuento: Decimal
    condiciones: List[Condicion] = field(default_factory=list)
```

dsl_interno > builders.py > PromocionBuilder > nombre

```python
class PromocionBuilder:
    def __init__(self, dsl: DSLSupermercado, codigo: int):
        self.dsl = dsl
        self.codigo = codigo
        self._nombre = None
        self._descuento = Decimal("0")
        self.condiciones = []

    def nombre(self, nombre: str) -> "PromocionBuilder":
        self._nombre = nombre
        return self
```

5. **Uso de un Árbol de Sintaxis Abstracta (AST) como capa intermedia:**

   Los parsers (Lark) no interactúan con la lógica de negocio directamente. Lo que hacen es leer el texto y devolver objetos de Python (dataclasses como CompraCommand o AgregarProducto).

   Esto separa completamente la sintaxis de la semántica. El parser solo se preocupa por entender el texto, y uno de los ejecutores se encarga de tomar ese AST y llamar al SupermercadoService.





6. **Interfaz Gráfica de Usuario con Streamlit:**

   Se desarrolló una interfaz web usando Streamlit para reemplazar la interacción por consola, facilitando la usabilidad.

   La GUI se diseñó como una capa de presentación, ya que consume exactamente el mismo SupermercadoService y los mismos parsers que el menú en consola. Se usó st.cache_resource para mantener el estado del supermercado vivo entre los clicks del usuario y CSS para hacer la interfaz de temática de supermercado.

```python
# --- Servicio cacheado: sobrevive recargas de página ---
@st.cache_resource
def get_service():
    s = SupermercadoService()
    cargar_configuracion_inicial(s)
    return s

try:
    service = get_service()
except Exception as e:
    st.error(f"Error al cargar config inicial: {e}")
    st.exception(e)
    st.stop()
```



Herramientas utilizadas

Para el desarrollo del sistema se seleccionaron un conjunto de herramientas y librerías que permitieran cumplir con los requerimientos de la consigna de manera eficiente, robusta y mantenible. A continuación, se detalla cada una de ellas y la justificación de su elección.

1. **Patrón de Diseño Builder (DSL Interno)**

   Se utilizó para construir la Fluent Interface del DSL interno (DSLSupermercado, ProductoBuilder, PromocionBuilder).

   La consigna exigía que el DSL interno permitiera encadenar métodos de forma declarativa (ej. .producto(1).nombre("Arroz")...). El patrón Builder permite separar la construcción de un objeto complejo de su representación final. Al combinarlo con la sintaxis de Python, logramos que el código sea autoexplicativo y fácil de reemplazar por otro DSL en el futuro.

2. **Lark para Análisis Sintáctico (DSL Externo):**

   Se eligió Lark para implementar los parsers de mantenimiento y compras, en lugar de PLY, porque permite definir la gramática de forma declarativa utilizando notación EBNF, que es similar a lo que vimos en Teoría de la Computación. Además, es más legible que programar las reglas a mano en funciones de Python.

   La característica principal que facilitó el desarrollo fue el uso de Transformers. Lark recorre el árbol sintáctico y, usando una clase en Python, transforma esos tokens directamente en nuestros objetos del dominio (ASTs).

   Por último, Lark nos da herramientas para el manejo de errores de sintáxis, indicando la línea y columna exacta en la que falló el archivo que se intenta subir.

3. **Pytest para Pruebas Unitarias**

   Se utilizó para escribir y ejecutar las pruebas unitarias del modelo de dominio, el motor de compras, los builders del DSL interno y los parsers/ejecutores de los DSL externos.

   Pytest nos permitió validar que el motor Greedy resolviera correctamente los 6 casos de ejemplo de la consigna, y que el servicio rechazara los datos inválidos. Este método es mucho más rápido que ir cargando cada archivo individualmente en la interfaz, garantizando así la confiabilidad del sistema.

4. **Docker**

   Evita problemas de versiones y dependencias entre las máquinas del equipo. Se utilizó para tener un entorno reproducible aislado.

   docker-compose.yml define tres servicios sobre la misma imagen: menú, gui y tests. El Dockerfile copia primero requirements.txt e instala dependencias antes de copiar el código, aprovechando la caché de capas.

Reglas de producción

1. Gramática del DSL de Mantenimiento

Esta gramática define cómo se estructuran los comandos para agregar, actualizar o eliminar productos y promociones:

dsl_externo > mantenimiento > parser.py > ...

```
GRAMMAR = r"""
start: comando+

comando: agregar_producto
       | actualizar_producto
       | eliminar_producto
       | agregar_promocion
       | actualizar_promocion
       | eliminar_promocion

// ==== Productos ====

agregar_producto:    "AGREGAR"    "PRODUCTO" INT campo_producto* "FIN"
actualizar_producto: "ACTUALIZAR" "PRODUCTO" INT campo_producto* "FIN"
eliminar_producto:   "ELIMINAR"   "PRODUCTO" INT

campo_producto: campo_nombre_prod | campo_precio_prod | campo_categoria_prod
campo_nombre_prod:    "NOMBRE"    STRING
campo_precio_prod:    "PRECIO"    NUMBER
campo_categoria_prod: "CATEGORIA" STRING

// ==== Promociones ====

agregar_promocion:    "AGREGAR"    "PROMOCION" INT elemento_promocion* "FIN"
actualizar_promocion: "ACTUALIZAR" "PROMOCION" INT elemento_promocion* "FIN"
eliminar_promocion:   "ELIMINAR"   "PROMOCION" INT

elemento_promocion: campo_nombre_promo | campo_descuento_promo | bloque_cuando
campo_nombre_promo:    "NOMBRE"    STRING
campo_descuento_promo: "DESCUENTO" NUMBER

bloque_cuando: "CUANDO" condicion+
condicion: "PRODUCTO" INT ("CANTIDAD" OPERADOR INT)?

// ==== Terminales ====

OPERADOR: ">=" | "="
NUMBER: /\d+(\.\d{1,2})?/
INT: /\d+/
STRING: /"[^"]*"/
```

Decisiones clave de esta gramática:

a. **Orden libre:**

El uso del operador * (cero o más) en campo_producto* [líneas 26 y 27] y elemento_promocion* [líneas 37 y 38] permite que los atributos como NOMBRE o PRECIO se escriban en cualquier orden.

b. **Lista de condiciones sin FIN:**

El token bloque_cuando se define como “CUANDO” condicion+, sin un deilimitador de cierre explícito [línea 45]. La lista de condiciones termina naturalmente cuando el parser encuentra un token que no es PRODUCTO o CANTIDAD (por ejemplo, un NOMBRE o el FIN del comando).

c. **Cantidad opcional**

El uso de ? en (“CANTIDAD” OPERADOR INT)? [línea 46] permite que si no se especifica la cantidad, el Transformer asuma el valor por defecto de 1 con operador =.

d. **Tipos de números**

Se diferencian INT de NUMBER, asegurando que los códigos y cantidades no acepten decimales pero los precios sí.

2. Gramática del DSL de Compras

Esta gramática define cómo se estructura un ticket de compra.

dsl_externo > compras > parser.py > ...

```
8
9    GRAMMAR = r"""
10   start: compra+
11
12   compra: "COMPRA" item+ "FIN"
13
14   item: "PRODUCTO" INT ("CANTIDAD" INT)?
15
16   INT: /\d+/
17
18   %import common.WS
19   %ignore WS
20   """
21
```

Decisiones clave de esta gramática:

a. Productos obligatorios:

   El uso de item+ (uno o más) garantiza sintácticamente que no se pueda haceruna compra vacía (un bloque COMPRA FIN tiraría error sintáctico, cumpliendo con la validación de la consigna).

b. Cantidad opcional:

   Al igual que en mantenimiento, el uso de ? permite que PRODUCTO 5 sea válido, por ejemplo, y el Transformer le asigne automáticamente la cantidad 1.

Conclusiones

El desarrollo de este sistema de gestión para un supermercado nos permitió integrar y aplicar de manera práctica los conceptos teóricos fundamentales de la materia, demostrando cómo una arquitectura bien diseñada puede resolver problemas complejos de negocio.

Consideramos que se logró construir una solución robusta, escalable y precisa. El sistema cumple con creces los requerimientos de la consigna, desde la carga inicial declarativa mediante el DSL interno, hasta la flexibilidad que brindan los DSL externos para la administración de datos y la simulación de compras. La decisión de separar estrictamente las responsabilidades (delegando la sintaxis a los parsers, la semántica al servicio central y los cálculos financieros al motor Greedy) resultó clave. Esto permitió que el sistema sea tolerante a fallos, reportando múltiples errores semánticos sin abortar la ejecución. Entendimos que un DSL es una herramienta poderosa para acercar la lógica del negocio a los usuarios o a otros sistemas, ocultando la complejidad técnica.

Por otro lado, enfrentarnos a la lógica del motor de compras nos obligó a salir del típico código y nos llevó a pensar en algoritmos que resolvieran combinaciones complejas y casos borde de manera óptima. Finalmente, el uso de Lark nos mostró cómo funciona el procesamiento de lenguajes "por dentro".

En conclusión, el trabajo no solo consolidó nuestros conocimientos de Python y patrones de diseño, sino que nos dio una perspectiva mucho más profesional sobre cómo diseñar software mantenible y de calidad.